

StG

Purpose

This app provides advanced steganography functions, especially:

- encrypt data with AES and embed encrypted data into .png image.
- extract encrypted data from .png image and decrypt it with AES.

For AES ciphering the app is using software from [this repository](#).

In simple words

This app allows to hide data into .png image and then extract it from there utilizing method names *steganography*.

Principle of this method is in mixing of data-to-hide with data of picture utilizing smart math.

Imagine you adding a salt to a water. It dissolves and become invisible, but it is still there. And you can extract it, if you, for example, evaporate water. This is what steganography does with data.

If you salted your water properly (i.e., not too hard), it's very difficult to determine (by eye) that it contains something else. Even more difficult to detect the presence of salt if you are adding it not to a pure water, but to a rich soup with vegetables, meat, spices and so on. Same way steganography hides the data in carrier. If amount of data to hide is not too big, it's extremely difficult to detect that an image carries something else.

Modern special steganalysis systems are able to detect *probability* of presence of hidden data in an image. Same way we can say "Salty!" if we try the soup. But we can not say *why* it's salty (because of salt or, for example, because of soy sauce).

To reduce possibility of such detection, **StG** app uses advanced methods of steganography. In a nutshell, the app analyzes carrier picture, determines zones where data hiding will be less visible, and hides data only there. Additionally, during hiding the app shuffles data in pseudo-random way, making it looks like a noise. Developing analogy with a soup, we each time inventing flavouring additive, perfectly matching natural taste of this specific soup.

Beside this **StG** provides additional level of protection. Before embedding, it encrypts data with AES encryption system (most advanced encryption standard in the world). This way, even if the attacker will detect the presence of steganographed data, he will not be able to decrypt it - without a password it is just a random noise. There is no methods to decipher AES.

Important notices

1. Data-to-carrier ratio for typical pictures is about 1.5%. I.e., **picture size of 1Mb can contains roughly 15kb of hidden data.**
2. Best choice for carrier pictures - **colorful and natural pictures with enough details**. Real photos with cosy interiors or beautiful landscapes will be the best option.
3. **WARNING!!!** "Under the hood" **StG** contains **military grade** encrypting system. If you

forget (or lost) the password, **NO ONE** will be able to decrypt your information! Brute-force time for deciphering AES-256 (even using quantum computers!) exceeds time of heat death of Universe.

Specifications

- **Available encryption standards:** AES-128, AES-192, AES-256
- **Steganography method:** +/-1 LSB, adaptive embedding, randomized selection
- **Carrier format:** .png
- **D/C ratio:** 0.015

Download

Latest version of StG app can be downloaded [here](#)

User interface

User interface of the app allows to:

- choose the operation
- set up data file, carrier image file and stego image file paths and parameters of encrypting/decrypting
- initialize operation
- receive feedback from the app

"Embed" tab

This tab allows to embed data to carrier image.

- **Data file** field - displays (and allows to input) path to file with data you want to *embed*.
- **Carrier image file** field - displays (and allows to input) path to file of carrier image *into which you want to embed* data.
- **Password** field - allows to input your password (at least 8 symbols, with least one capital letter, one digit and one special symbol).
- **Encryption mode** combo box - displays selected AES mode and allows to select other available (*AES-128, AES-192 or AES-256*).
- **Stego image file** field - displays (and allows to input) path to file of stego image (which is the *result of embedding operation*).
- **Embed** button - initialize embedding operation. Before embedding, system validates all parameters. If something is wrong, system will output a report in "Status" field in the lower left corner of the window.

If data volume is too big for this carrier, system will report status "StGLib::Embed failed." In this case choose bigger carrier image.

"Extract" tab

This tab allows to extract data from stego image.

- **Stego image file** field - displays (and allows to input) path to file of stego image, which contains *encrypted data to extract*.
- **Password** field - allows to input your password (at least 8 symbols, with least one capital letter, one digit and one special symbol).
- **Encryption mode** combo box - displays selected AES mode and allows to select other available (*AES-128, AES-192 or AES-256*).
- **Data file** field - displays (and allows to input) path to file with data (which is the *result of extracting operation*).
- **Extract** button - initialize extract operation. Before extracting, system validates all parameters. If something is wrong, system will output a report in "Status" field in the lower left corner of the window.

In fields **Data file**, **Carrier image file** and **Stego image file** of both tabs you can type the path to appropriate files by yourself or select it via GUI dialogue through "*Browse*" button near the end of the field.

Notice: system does not touch any info in source files (read-only access), and does not allow to overwrite destination files.

Roadmap of further development

- Improve base functional
 - Reduce distributive size
 - Stego density control
 - Consume different image formats as carrier
 - Mobile OS versions
 - Android version
 - iOS version (donations for MacBook & iPhone required!)
 - Stego-messenger **Narodovolets** - distributed, secured, free messenger for all main platforms
 - Advanced stego algorithms (ISGANs, DCT, DWT, RDH)
-

Questions

Feel free to ask me if you have any questions or offers.

Legal notice

This software is distributing under MIT/BSD-like license. Full terms are available in the [License](#)

file.

The project uses **OpenCV** [Open Source Computer Vision Library](#). The OpenCV License requirements completed. Copyright Notices included at [OpenCV license.txt](#).

Libertatian Stegosaurus Randall, who are a mascot of the project, designed by author of the project. Logo image generated using [ChatGPT 5.5](#).

App icon was created from logo image using [PNG-to-ICO](#).

Support & donations

If you like this software, please, support me with donations. This will allow me to spend more time on improvement of my projects. Any help will count.

VISA card (Halyk bank, Worldwide): 4003 0351 5709 6871

MIR card (T-bank, Russia only): 2200 7021 5287 4603

Receipient: Daniil Galakhov